

UNIVERSIDAD AUTONOMA DE TAMAULIPAS
FACULTAD DE INGENIERIA TAMPICO

PROGRAMACION DE SISTEMAS DE BASE I
MUÑOZ QUINTERO DANTE ADOLFO

8-M

PROYECTO INTEGRADOR – PROYECTO DE ANALIZADOR
LEXICO
DRONE SCRIPT

INTEGRANTES:

TORRES ALVINEDA MANUEL DE JESUS

TURRUBIATES MEJIA GILBERTO

VEGA ANGUIANO ALEXA FERNANDA

GITHUB : [ManuelAlvi25/Proyecto-Analizador-
Lexico_DroneScript](https://github.com/ManuelAlvi25/Proyecto-Analizador-Lexico_DroneScript)

[alexavegaanguiano-wq/Proyecto-Analizador-lexico](https://github.com/alexavegaanguiano-wq/Proyecto-Analizador-lexico): Proyecto
de la materia sistemas de base 1

[GilbertoTurrubiates/analizador-lexico-DRONESCRIP](https://github.com/GilbertoTurrubiates/analizador-lexico-DRONESCRIP)

INDICE

Introducción	3
Objetivos	4
Especificación del lenguaje	5
Implementación	7
Demostración funcional	20
Conclusiones	22
Referencias.....	23

Introducción

Contexto del Lenguaje DroneScript es un lenguaje de programación personalizado, diseñado de manera específica para el control y la automatización de drones. El proyecto actual, DroneScript Lexer, representa la primera fase fundamental para la construcción de este lenguaje: un analizador léxico robusto construido completamente desde cero mediante Python. Este sistema nace en un contexto académico como proyecto integrador para un curso llamado programación de sistemas de base I, buscando aplicar conceptos teóricos en una herramienta funcional y con calidad profesional.

Motivación y Dominio que busca resolver El dominio principal de DroneScript es la robótica y los vehículos aéreos no tripulados. El problema que busca resolver es la complejidad de programar instrucciones para estos dispositivos, ofreciendo a cambio un vocabulario altamente especializado e intuitivo.

Para lograrlo, el lenguaje abstrae las operaciones complejas de los drones en más de 127 palabras clave organizadas en categorías semánticas precisas. Esto permite a los programadores manejar directamente y de forma estructurada los siguientes aspectos del dominio:

- Control de Vuelo y Movimiento: Instrucciones directas como despegar, aterrizar, orbitar o mantener la altura.
- Navegación y Entorno: Gestión de coordenadas GPS, rutas y factores ambientales como el viento o la lluvia.
- Interacción con Hardware: Control de sensores (ej. lidar, cámaras térmicas, ultrasonido), gestión de energía y niveles de batería.
- Autonomía y Seguridad: Implementación de modos autónomos de inteligencia artificial, prevención de colisiones, autenticación y manejo de eventos del entorno.

Además de la motivación funcional del lenguaje, existe una fuerte motivación académica: proporcionar una plataforma para entender en la práctica las fases de compilación, el diseño de arquitecturas modulares (preparadas para futuras fases como el parser y la construcción del AST), el reconocimiento de patrones y el manejo avanzado de errores léxicos.

Objetivos

Objetivo General

Desarrollar un analizador léxico robusto y completo desde cero para DroneScript, un lenguaje personalizado orientado al control de drones y robots autónomos, sirviendo como proyecto práctico para el curso de Programación de sistemas de base 1 y estableciendo una arquitectura modular sólida para las futuras fases de compilación.

Objetivos Específicos

- **Fase de Especificación del Lenguaje:**
 - Definir el vocabulario de DroneScript mediante el reconocimiento de más de 127 palabras reservadas, categorizándolas semánticamente según su rol (por ejemplo: control de vuelo, sensores, inteligencia artificial, navegación GPS).
 - Establecer las reglas léxicas para la identificación precisa de identificadores, números (enteros y flotantes), cadenas de texto (con soporte para secuencias de escape), operadores y delimitadores.
- **Fase de Implementación del Lexer:**
 - Construir el analizador utilizando Python puro (sin dependencias externas), aplicando un diseño modular e independiente que facilite su reutilización y acoplamiento en fases futuras (como el parser sintáctico y el árbol AST).
 - Desarrollar un sistema de manejo de errores avanzado que detecte fallos (como caracteres desconocidos o cadenas no cerradas) y que cuente con recuperación de errores, permitiendo acumular múltiples problemas sin detener el análisis en el primer fallo.
 - Proveer salidas estructuradas y flexibles que faciliten el consumo de los datos en fases posteriores, ofreciendo formatos como tabla ASCII, JSON y salidas simples.
- **Fase de Pruebas y Validación:**
 - Ejecutar casos de prueba exhaustivos con archivos válidos e inválidos para asegurar que todos los componentes léxicos generen los tokens esperados de forma confiable.
 - Validar la precisión del lexer verificando que la información de ubicación (línea y columna exactas) y el contexto visual de los errores se reporten de manera correcta durante el análisis.

Especificación del lenguaje

Nombre del Lenguaje y Justificación El lenguaje se llama DroneScript. Su creación se justifica por la necesidad de contar con una herramienta especializada para el control y automatización de drones. A diferencia de los lenguajes de propósito general, DroneScript proporciona un vocabulario de alto nivel (con más de 127 palabras reservadas) que abstrae la complejidad del hardware, permitiendo a los desarrolladores programar rutas, sensores, validaciones de seguridad y modos de inteligencia artificial de forma intuitiva, semántica y directa.

Descripción informal de la sintaxis con ejemplos de código ilustrativos La sintaxis de DroneScript es estructurada e imperativa. Utiliza palabras clave en español para definir acciones, estructuras de control de flujo estándar (como condicionales y ciclos), delimitadores clásicos (llaves para bloques de código, paréntesis para expresiones) y terminadores de instrucción (punto y coma). Además, soporta comentarios de línea y de bloque para la documentación del código.

Ejemplo de código: Secuencia de despegue y validación de batería

```
// Iniciar rutina de vuelo
despegar;

si (nivel_bateria > 20) {
    mover norte;
    activar_camara;
} sino {
    aterrizar;
}
```

Ejemplo de código: Uso de variables, sensores y ciclos

```
/* Escaneo del entorno
   usando el sensor lidar */
mientras (detectar_obstaculo == falso) {
    avanzar;
    escanear;
}
detener;
```

Tabla de Tokens A continuación, se detalla la clasificación de los componentes léxicos del lenguaje, incluyendo su categoría semántica, descripción y la expresión regular o patrón utilizado para su reconocimiento:

Categoría	Descripción	Patrón / Expresión Regular
KEYWORD (14 Categorías)	Palabras reservadas agrupadas semánticamente (CONTROL, MOVEMENT, DIRECTION, GPS, SENSOR, CAMERA, BATTERY,	despegar mover norte si ... (Cadenas literales exactas para las 127 palabras)

	<i>CONDITIONAL, COMMUNICATION, SECURITY, ENVIRONMENT, AI, DATA_TYPE, EVENT)</i>	
OPERATOR_ARITHMETIC	Operadores para cálculos matemáticos	\+ - * / %
OPERATOR_COMPARISON	Operadores relacionales para condiciones	< > <= >= == !=
OPERATOR_LOGICAL	Operadores lógicos (AND, OR, NOT)	&& !
OPERATOR_ASSIGNMENT	Asignación de valores a variables	=
DELIMITER_PAREN	Paréntesis para agrupar expresiones lógicas/matemáticas	\(\)
DELIMITER_BRACE	Llaves para definir la apertura y cierre de bloques de código	\{ \}
DELIMITER_BRACKET	Corchetes para colecciones o arreglos	\[\]
DELIMITER_PUNCTUATION	Signos de puntuación, incluyendo fin de instrucción y separadores	; , :
LITERAL_NUMBER	Números enteros positivos y negativos	\d+
LITERAL_FLOAT	Números decimales o de punto flotante	\d+\.\d+
LITERAL_STRING	Cadenas de texto con soporte para secuencias de escape (\n, \t, \", \\)	"[^\"]*"(\\".["\s\S]*?")"
LITERAL_IDENTIFIER	Identificadores para variables definidas por el usuario	[a-zA-Z_][a-zA-Z0-9_]*
COMENTARIOS	Ignorados por el lexer: Comentarios de línea (//) y de bloque (/* */)	//.* /\s\S]*?\/

Implementación

4.1 Definición de tokens y palabras reservadas

La implementación del lexer de DroneScript se basa en el reconocimiento de una amplia variedad de tokens que construyen el lenguaje. Se definen **127 palabras clave reservadas**, las cuales están organizadas en **14 categorías semánticas explícitas** que reflejan su rol dentro del control de drones. Estas categorías incluyen instrucciones de CONTROL (ej. *despegar*, *aterrizar*), MOVEMENT (ej. *mover*, *rotar*), SENSOR (ej. *escanear*, *lidar*), GPS, CAMERA, entre otras.

Además de las palabras reservadas, se definen tokens para:

- **Identificadores:** Nombres de variables válidos e inválidos.
- **Literales:** Números (enteros y de punto flotante) y Cadenas de texto (strings con soporte para secuencias de escape).
- **Operadores:** Aritméticos, de comparación, lógicos y de asignación (simples y multi-carácter).
- **Delimitadores:** Paréntesis, llaves, corchetes y signos de puntuación.
- **Comentarios:** De línea simple (*//*) y de bloque (*/* */*).

Clase de tokens.py

```
from enum import Enum, auto
from dataclasses import dataclass
from typing import Optional, Any

class TokenType(Enum):
    """
    """
    # Palabras clave
    KEYWORD = auto()

    # Literales
    NUMBER = auto()
    FLOAT = auto()
    STRING = auto()
    IDENTIFIER = auto()

    # Operadores
    OPERATOR = auto()

    # Delimitadores
    DELIMITER = auto()
```

```
# Especiales
EOF = auto()
UNKNOWN = auto()

class TokenCategory(Enum):
    # Control general
    CONTROL = auto()

    # Movimiento
    MOVEMENT = auto()

    # Direcciones
    DIRECTION = auto()

    # GPS y navegación
    GPS = auto()

    # Sensores
    SENSOR = auto()

    # Cámara y visión
    CAMERA = auto()

    # Batería y energía
    BATTERY = auto()

    # Condicionales y ciclos
    CONDITIONAL = auto()

    # Comunicación
    COMMUNICATION = auto()

    # Seguridad
    SECURITY = auto()

    # Clima y entorno
    ENVIRONMENT = auto()

    # IA y autonomía
    AI = auto()

    # Tipos de datos
    DATA_TYPE = auto()
```



```
# Eventos automáticos
EVENT = auto()

# Operadores
OPERATOR_ARITHMETIC = auto()
OPERATOR_COMPARISON = auto()
OPERATOR_LOGICAL = auto()
OPERATOR_ASSIGNMENT = auto()

# Delimitadores
DELIMITER_PAREN = auto()      # ( )
DELIMITER_BRACE = auto()      # { }
DELIMITER_BRACKET = auto()    # [ ]
DELIMITER_PUNCTUATION = auto() # ; , :

# Literales
LITERAL_IDENTIFIER = auto()
LITERAL_NUMBER = auto()
LITERAL_FLOAT = auto()
LITERAL_STRING = auto()

# Otros
UNKNOWN = auto()
```

4.2 Reglas léxicas (expresiones regulares o gramática)

Para el reconocimiento de los patrones, se tomó la **decisión técnica de construir un lexer manual en Python puro** en lugar de generar uno automáticamente mediante herramientas como ANTLR. Esta decisión obedece al propósito académico del curso de **Programación de sistemas de base 1**, ya que implementar las reglas léxicas manualmente otorga un mayor control y una mejor comprensión del proceso de tokenización y reconocimiento de patrones. No obstante, las reglas léxicas implementadas mantienen la compatibilidad con una especificación formal (cuyos detalles gramaticales de ANTLR se contemplan en la documentación del proyecto) para posibles integraciones futuras.

DroneScript.g4

```
// ===== CANALES (Skipear automáticamente) =====
channels {
|   WHITESPACE_CHANNEL,
|   COMMENTS_CHANNEL
}

// ===== WHITESPACE - Se ignora automáticamente =====
WS: [ \t\r\n]+ -> channel(HIDDEN);

// ===== COMENTARIOS =====
LINE_COMMENT: '//' ~[\r\n]* -> channel(HIDDEN);
BLOCK_COMMENT: '/*' .*? '*/' -> channel(HIDDEN);

// ===== PALABRAS CLAVE - CONTROL GENERAL =====
KW_DESPEGAR: 'despegar';
KW_ATERRIZAR: 'atterrizar';
KW_INICIAR_MISION: 'iniciar_mision';
KW_FINALIZAR_MISION: 'finalizar_mision';
KW_PAUSAR_MISION: 'pausar_mision';
KW_REANUDAR_MISION: 'reanudar_mision';
KW_REGRESAR_BASE: 'regresar_base';
KW_APAGAR: 'apagar';
KW_REINICIAR: 'reiniciar';
KW_DETENER: 'detener';
KW_EMERGENCIA: 'emergencia';
```

(código restante en el propio archivo)

4.3 Lógica del analizador léxico

La lógica central del analizador léxico consiste en recorrer el código fuente carácter por carácter para agruparlos en tokens válidos. El sistema está construido mediante un **diseño modular** con componentes independientes y sin requerir dependencias externas de Python.

Durante el procesamiento, el lexer no solo identifica el token, sino que extrae **información semántica completa** para cada uno de ellos, guardando:

- El tipo general (TokenType como KEYWORD, NUMBER, etc.).
- La categoría semántica específica (TokenCategory como CONTROL, DIRECTION, etc.).
- El lexema original y su valor literal.
- La ubicación exacta en el código.

Finalmente, el analizador procesa estos datos y ofrece una **salida estructurada y flexible** (en formatos como tabla ASCII, JSON o salida simple) que facilita el consumo de los tokens por parte de futuras fases del compilador, como el parser sintáctico o el árbol de derivación (AST).

lexer.py

```
from typing import Tuple, List
from src.tokens import Token, TokenType, TokenCategory
from src.errors import ErrorCollector, ErrorType
from src import utils

class DroneLexer:
    """
    Analizador léxico para el lenguaje DroneScript.
    Convierte texto fuente en una lista de tokens.
    """

    def __init__(self, source: str, filename: str = "<stdin>"):
        """
        Inicializa el lexer.

        Args:
            source: Código fuente a analizar
            filename: Nombre del archivo (para reportes)
        """
        self._input = source
        self._filename = filename
        self._position = 0
        self._line = 1
        self._column = 1
        self._tokens: List[Token] = []
        self._errors = ErrorCollector()
        self._line_starts: List[int] = [0] # Índices de inicio de línea

    # ===== MÉTODOS PRINCIPALES =====

    def tokenize(self) -> Tuple[List[Token], ErrorCollector]:
        """
        Ejecuta el análisis léxico completo.

        Returns:
            Tupla (lista de tokens, recolector de errores)
        """
        while not self._at_end():
```

```

        # Intentar reconocer siguiente token
        if not self._scan_token():
            # Carácter desconocido
            self._report_unknown_character()

    return self._tokens, self._errors

# ===== MÉTODOS DE LECTURA =====

def _peek(self, offset: int = 0) -> str:
    """Mira carácter sin consumir."""
    pos = self._position + offset
    if pos >= len(self._input):
        return '\0'
    return self._input[pos]

def _advance(self) -> str:
    """Consume y retorna carácter, actualiza línea/columna."""
    if self._at_end():
        return '\0'

    ch = self._input[self._position]
    self._position += 1

    if ch == '\n':
        self._line += 1
        self._column = 1
        self._line_starts.append(self._position)
    else:
        self._column += 1

    return ch

def _match(self, expected: str) -> bool:
    """Consume carácter si coincide con expected."""
    if self._peek() == expected:
        self._advance()
        return True
    return False

def _at_end(self) -> bool:
    """Verifica si se llegó al final de entrada."""
    return self._position >= len(self._input)

# ===== MÉTODOS DE ESCANEO DE TOKENS =====

```

```
def _scan_token(self) -> bool:
    """
    Intenta reconocer el siguiente token.
    Retorna True si reconoció algo, False si carácter desconocido.
    """
    ch = self._peek()

    # Espacios en blanco
    if ch.isspace():
        self._skip_whitespace()
        return True

    # Comentarios
    if ch == '/' and self._peek(1) == '/':
        self._skip_line_comment()
        return True

    if ch == '/' and self._peek(1) == '*':
        self._skip_block_comment()
        return True

    # Strings
    if ch == '"':
        self._scan_string()
        return True

    # Números
    if ch.isdigit():
        self._scan_number()
        return True

    # Identificadores y palabras clave
    if ch.isalpha() or ch == '_':
        self._scan_keyword_or_identifier()
        return True

    # Operadores
    if utils.is_operator(ch):
        self._scan_operator()
        return True

    # Delimitadores
    if utils.is_delimiter(ch):
        self._scan_delimiter()
```

```

        return True

    # Carácter desconocido
    return False

def _skip_whitespace(self) -> None:
    """Salta espacios en blanco."""
    while not self._at_end() and self._peek().isspace():
        self._advance()

def _skip_line_comment(self) -> None:
    """Salta comentario de línea (// ... \n)."""
    self._advance() # primer /
    self._advance() # segundo /
    while not self._at_end() and self._peek() != '\n':
        self._advance()
    if not self._at_end():
        self._advance() # consumir \n

def _skip_block_comment(self) -> None:
    """Salta comentario de bloque (/* ... */)."""
    start_line = self._line
    start_col = self._column

    self._advance() # /
    self._advance() # *

    while not self._at_end():
        if self._peek() == '*' and self._peek(1) == '/':
            self._advance() # *
            self._advance() # /
            return
        self._advance()

    # Comentario no cerrado
    self._errors.add_error(
        ErrorType.UNTERMINATED_COMMENT,
        start_line,
        start_col,
        '/*',
        'Comentario de bloque no cerrado'
    )

def _scan_string(self) -> None:
    """Escanea string con comillas dobles."""

```

```

start_line = self._line
start_col = self._column
self._advance() # consumir "

value = []

while not self._at_end() and self._peek() != '":
    if self._peek() == '\n':
        # String no cerrado en la línea
        self._errors.add_error(
            ErrorType.UNTERMINATED_STRING,
            start_line,
            start_col,
            '',
            'String no cerrado (newline encontrado)'
        )
        return

    if self._peek() == '\\':
        self._advance()
        if self._at_end():
            self._errors.add_error(
                ErrorType.UNTERMINATED_STRING,
                start_line,
                start_col,
                '',
                'String no cerrado (fin de archivo)'
            )
            return
        value.append(self._advance())
    else:
        value.append(self._advance())

if self._at_end():
    # String no cerrado al fin de archivo
    self._errors.add_error(
        ErrorType.UNTERMINATED_STRING,
        start_line,
        start_col,
        '',
        'String no cerrado (fin de archivo)'
    )
    return

self._advance() # consumir cierre "

```

```

# Procesar escapes
raw_string = ''.join(value)
processed, is_valid = utils.process_escape_sequences(raw_string)

if not is_valid:
    self._errors.add_error(
        ErrorType.INVALID_ESCAPE_SEQUENCE,
        start_line,
        start_col,
        '',
        f'Secuencia de escape inválida en string'
    )

self._add_token(
    TokenType.STRING,
    TokenCategory.LITERAL_STRING,
    f'"{raw_string}"',
    processed
)

def _scan_number(self) -> None:
    """Escanea números enteros y flotantes."""
    start_col = self._column
    value = []

    # Escanear parte entera
    while not self._at_end() and self._peek().isdigit():
        value.append(self._advance())

    # Verificar punto decimal
    if not self._at_end() and self._peek() == '.' and
self._peek(1).isdigit():
        value.append(self._advance()) # punto
        while not self._at_end() and self._peek().isdigit():
            value.append(self._advance())

        number_str = ''.join(value)
        self._add_token(
            TokenType.FLOAT,
            TokenCategory.LITERAL_FLOAT,
            number_str,
            float(number_str)
        )
    else:

```



```

        number_str = ''.join(value)
        self._add_token(
            TokenType.NUMBER,
            TokenCategory.LITERAL_NUMBER,
            number_str,
            int(number_str)
        )

def _scan_keyword_or_identifier(self) -> None:
    """Escanea palabras clave e identificadores."""
    value = []

    while not self._at_end() and (self._peek().isalnum() or self._peek()
== '_'):
        value.append(self._advance())

    word = ''.join(value)

    # Verificar si es palabra clave
    category = utils.get_keyword_category(word)
    if category:
        self._add_token(TokenType.KEYWORD, category, word)
    else:
        # Es identificador
        self._add_token(
            TokenType.IDENTIFIER,
            TokenCategory.LITERAL_IDENTIFIER,
            word
        )

def _scan_operator(self) -> None:
    """Escanea operadores (simples y multi-carácter)."""
    # Intentar operadores de 2 caracteres primero
    two_char = self._peek() + self._peek(1)
    if two_char in utils.MULTICHAR_OPERATORS:
        self._advance()
        self._advance()
        category = utils.get_operator_category(two_char)
        self._add_token(TokenType.OPERATOR, category, two_char)
    else:
        # Operador de 1 carácter
        ch = self._advance()
        category = utils.get_operator_category(ch)
        self._add_token(TokenType.OPERATOR, category, ch)

```

```

def _scan_delimiter(self) -> None:
    """Escanea delimitadores."""
    ch = self._advance()
    category = utils.get_delimiter_category(ch)
    self._add_token(TokenType.DELIMITER, category, ch)

# ===== MÉTODOS AUXILIARES =====

def _add_token(self, token_type: TokenType, category: TokenCategory,
               lexeme: str, literal_value=None) -> None:
    """Agrega un token a la lista."""
    token = Token(
        type=token_type,
        category=category,
        lexeme=lexeme,
        literal_value=literal_value,
        line=self._line,
        column=self._column - len(lexeme)
    )
    self._tokens.append(token)

def _report_unknown_character(self) -> None:
    """Reporta carácter desconocido como error."""
    ch = self._advance()
    self._errors.add_error(
        ErrorType.UNKNOWN_CHARACTER,
        self._line,
        self._column - 1,
        ch,
        f"Símbolo desconocido '{ch}'"
    )

def _get_line_content(self, line_num: int) -> str:
    """Obtiene el contenido de una línea específica."""
    if line_num < 1 or line_num - 1 >= len(self._line_starts):
        return ""

    start = self._line_starts[line_num - 1]
    end = self._line_starts[line_num] if line_num <
len(self._line_starts) else len(self._input)

    return self._input[start:end].rstrip('\n')

# ===== MÉTODOS DE ACCESO A RESULTADOS =====

```

```

def get_tokens(self) -> List[Token]:
    """Retorna la lista de tokens."""
    return self._tokens.copy()

def get_errors(self) -> ErrorCollector:
    """Retorna el recolector de errores."""
    return self._errors

def has_errors(self) -> bool:
    """Retorna True si hay errores."""
    return self._errors.has_errors()

```

4.4 Manejo y reporte de errores (con número de línea y columna)

El lexer implementa un **sistema de manejo robusto de errores** que garantiza que el proceso de análisis sea confiable. Sus características principales incluyen:

- **Detección exhaustiva:** Identifica caracteres desconocidos, strings no cerrados, comentarios sin cerrar y secuencias de escape inválidas.
- **Recuperación de errores:** En lugar de detener su ejecución al encontrar el primer fallo, el lexer tiene la capacidad de continuar analizando el resto del código. Esto permite **acumular múltiples errores** en una sola pasada, proporcionando una interfaz uniforme donde el usuario puede ver todos los problemas de una vez.
- **Reportes precisos:** Cada error detectado se reporta con su ubicación exacta (**número de línea y columna precisas**) y se acompaña de un **contexto visual** para que el desarrollador identifique rápidamente la falla en el código fuente.

errors.py

```

from enum import Enum, auto
from dataclasses import dataclass
from typing import List, Optional

class ErrorType(Enum):
    """Tipos de errores léxicos posibles."""
    UNKNOWN_CHARACTER = auto()
    UNTERMINATED_STRING = auto()
    UNTERMINATED_COMMENT = auto()
    INVALID_ESCAPE_SEQUENCE = auto()
    INVALID_FLOAT_FORMAT = auto()

@dataclass
class LexerError:

```

Demostración funcional

Las siguientes capturas muestran las ejecuciones desde la terminal.

5.1 Ejecución con un programa valido y su lista de tokens generada

Desde el archivo tests/valid/simple.drs

```
// Archivo simple.drs - Ejemplo mínimo del lexer

despegar;
aterrizar;
```

Ejecución y Salida

```
=====
TOKEN ANALYSIS REPORT
=====
File: tests\valid\simple.drs
Total tokens: 4
Status: ✓ OK (no errors)

TYPE          | LEXEME          | LINE | COL | CATEGORY
-----
KEYWORD       | despegar        | 3    | 1   | CONTROL
DELIMITER     | ;               | 3    | 9   | DELIMITER_PUNCTUATION
KEYWORD       | aterrizar       | 4    | 1   | CONTROL
DELIMITER     | ;               | 4    | 10  | DELIMITER_PUNCTUATION
```

5.2 Ejecucion con entradas que contengan errores léxicos y los mensajes producidos

Desde el archivo tests/invalid/broken_strings.drs

```
// Archivo con strings no cerrados

despegar;
nombre = "New York;
activar_camara;
mensaje = "Error en string;
aterrizar;
```

Ejecución y salida

```

PS C:\Users\Manuel\Documents\Proyecto Analizador Lexico> python -m src.main tests/invalid/broken_strings.drs
=====
TOKEN ANALYSIS REPORT
=====
File: tests/invalid/broken_strings.drs
Total tokens: 10
Status: X ERRORS DETECTED

=====
TYPE      | LEXEME          | LINE | COL | CATEGORY
=====
KEYWORD   | despegar        | 3    | 1   | CONTROL
DELIMITER | ;               | 3    | 9   | DELIMITER_PUNCTUATION
IDENTIFIER| nombre          | 4    | 1   | LITERAL_IDENTIFIER
OPERATOR  | =               | 4    | 8   | OPERATOR_ASSIGNMENT
KEYWORD   | activar_camara  | 5    | 1   | CAMERA
DELIMITER | ;               | 5    | 15  | DELIMITER_PUNCTUATION
IDENTIFIER| mensaje         | 6    | 1   | LITERAL_IDENTIFIER
OPERATOR  | =               | 6    | 9   | OPERATOR_ASSIGNMENT
KEYWORD   | aterrizar       | 7    | 1   | CONTROL
DELIMITER | ;               | 7    | 10  | DELIMITER_PUNCTUATION

=====
LEXICAL ERRORS DETECTED (2)
=====

[ERROR 1] UNTERMINATED_STRING
Line 4, Column 10
Character: ''
Message: String no cerrado (newline encontrado)

[ERROR 2] UNTERMINATED_STRING
Line 6, Column 11
Character: ''
Message: String no cerrado (newline encontrado)

```

5.3 Al menos con un caso con identificadores inválidos y/o símbolos desconocidos

```

tests > invalid > ❏ unknown_chars.drs
1 // Archivo con caracteres desconocidos
2
3 despegar;
4 mover norte 10#;
5 rotar @90;
6 aterrizar$;

```

Ejecución y salida

```

PS C:\Users\Manuel\Documents\Proyecto Analizador Lexico> python -m src.main tests/invalid/unknown_chars.drs
=====
TOKEN ANALYSIS REPORT
=====
File: tests\invalid\unknown_chars.drs
Total tokens: 11
Status: X ERRORS DETECTED

=====
TYPE      | LEXEME      | LINE | COL | CATEGORY
-----|-----|-----|-----|-----
KEYWORD   | despegar    | 3    | 1   | CONTROL
DELIMITER | ;           | 3    | 9   | DELIMITER_PUNCTUATION
KEYWORD   | mover       | 4    | 1   | MOVEMENT
KEYWORD   | norte       | 4    | 7   | DIRECTION
NUMBER    | 10          | 4    | 13  | LITERAL_NUMBER
DELIMITER | ;           | 4    | 16  | DELIMITER_PUNCTUATION
KEYWORD   | rotar       | 5    | 1   | MOVEMENT
NUMBER    | 90          | 5    | 8   | LITERAL_NUMBER
DELIMITER | ;           | 5    | 10  | DELIMITER_PUNCTUATION
KEYWORD   | aterrizar   | 6    | 1   | CONTROL
DELIMITER | ;           | 6    | 11  | DELIMITER_PUNCTUATION

=====
LEXICAL ERRORS DETECTED (3)
=====

[ERROR 1] UNKNOWN_CHARACTER
Line 4, Column 15
Character: '#'
Message: Símbolo desconocido '#'

[ERROR 2] UNKNOWN_CHARACTER
Line 5, Column 7
Character: '@'
Message: Símbolo desconocido '@'

```

Conclusiones

- 1.- Manuel de Jesús Torres Alvineda:** Hacer este analizador desde cero usando Python puro sin librerías externas fue un reto, pero me sirvió mucho para entender realmente cómo la computadora lee el texto letra por letra para formar los tokens. Al final, logramos crear un programa que procesa nuestro código sin depender de herramientas mágicas.
- 2.- Gilberto Turrubiates Mejía:** Para mí, lo más interesante del proyecto fue programar el manejo de errores. Me di cuenta de que es súper útil que el programa no se congele o se detenga al primer carácter raro que encuentra, sino que siga leyendo el archivo y al final te diga exactamente en qué línea y columna te equivocaste para corregirlo más rápido.
- 3.- Alexa Fernanda Vega Anguiano:** Con este proyecto para la materia de Programación de sistemas de base 1, me quedó mucho más claro cómo es el primer paso para crear un lenguaje real. Acomodar las 127 palabras clave en categorías sencillas como movimiento o sensores nos deja el terreno preparado y bien ordenado para cuando nos toque hacer las siguientes fases del compilador.

Referencias

ANTLR4 Documentation (<https://antlr.org/>)

ANTLR4 Python Runtime (<https://github.com/antlr/antlr4/tree/master/runtime/Python3>)